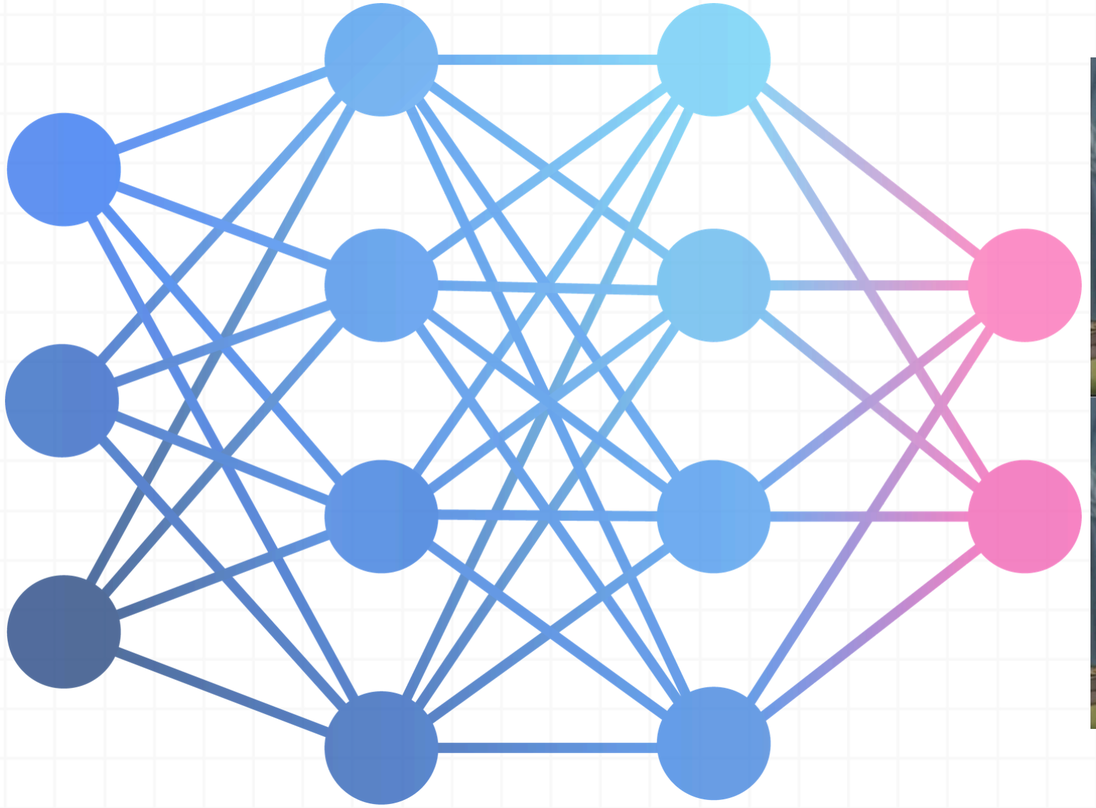
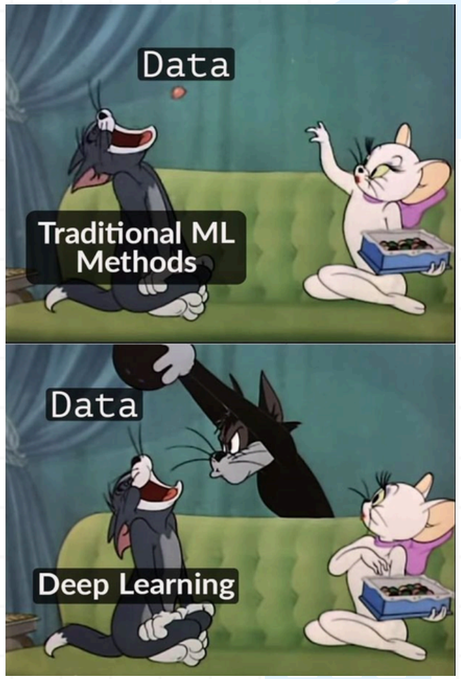
Institute of Technology of Cambodia
Department of Applied
Mathematics and Statistics

Forward Propagation, Step by Step

– How a deep neural network turns data into a prediction





Artifficial Intelligence
Lecturer: VANN Visal

Presented By: Group 1

Core Member



VON NAROTH



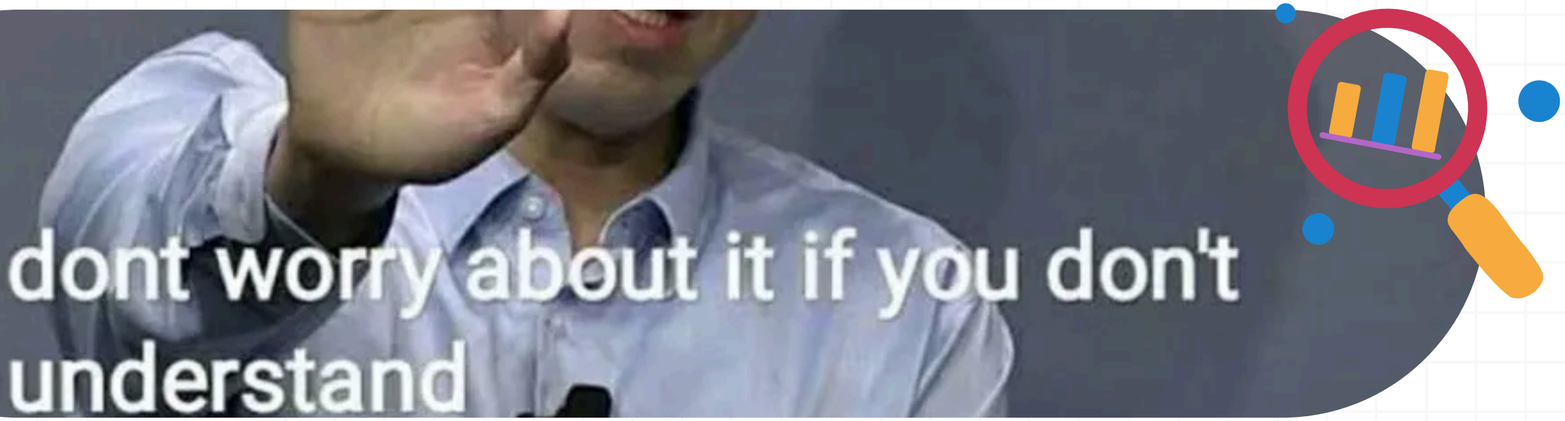
TAN VOUCHLEANG



KIM PUTDARARITH



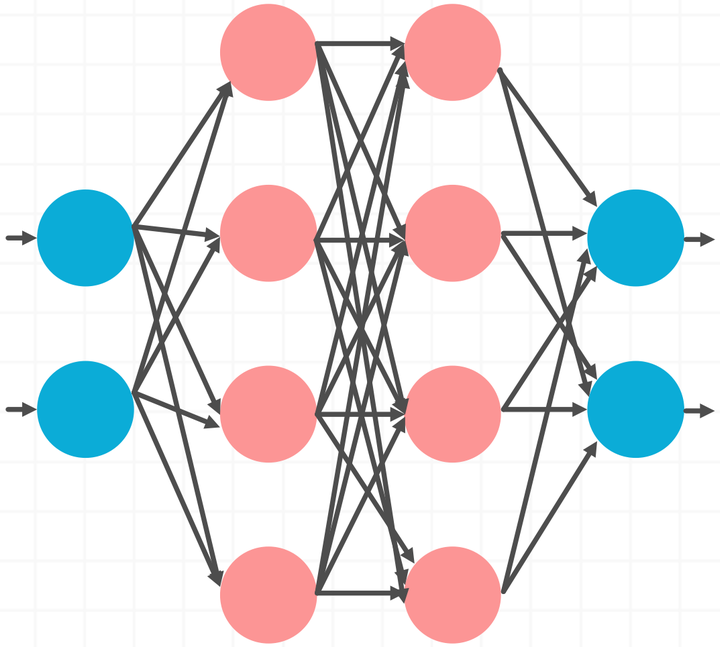
TUY CHAMROEUN



Contents

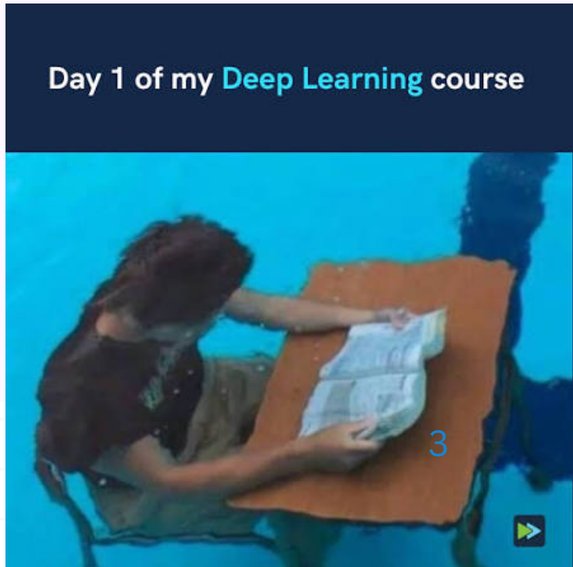
1. The plan: what we build and how data flows

2. Initialization: how the network starts



3. Linear step and activation: what one layer does

4. The whole forward pass: putting it all together



Library Set Up



```
import torch
import matplotlib.pyplot as plt

torch.set_printoptions(precision=4, sci_mode=False)
print("PyTorch", torch.__version__)
```

```
PyTorch 2.11.0+cu130
```

+cu130

built in CUDA 13.0 support for NVIDIA GPU

- **import torch**: Getting PyTorch library for using in building neural network and machine learning part.
- **import matplotlib.pyplot as plt**: Getting matplotlib library to show graph and chart visually.
- **torch.set_printoptions**: controls how numbers are displayed.
  - **precision=4**: displays 4 decimal number places.
  - **sci_mode=False**: not showing scentific notation in output; eg: 1.000e-6 to 0.0000.



1.The Plan

INPUT X

Training examples

→

FORWARD

Calculate Z, activation A

→

COST J

Compare prediction with Y

→

BACKWARD

Calculate gradients dW, db

→

UPDATE

Adjust W and b

Forward pass →

X → [LINEAR → ReLU] × (L-1) → LINEAR → Sigmoid → AL → cost

- Turn the input into a prediction (AL) and measure how wrong it is with the cost

Backward pass ←

grads ← [LINEAR ← ReLU] × (L-1) ←LINEAR ← Sigmoid ← dZ[L] = AL - Y

- Find how much each W and b caused the error (dW, db), so we can update them

During Forward Propagation, values of A, parameters and Z are stored in cache, so that Backward Propagation can use those value to calculate gradient.

The 9 Core Functions

1 initialize_parameters_deep()

Creates $\mathbf{W}^{[l]}$ and $\mathbf{b}^{[l]}$ for every layer
$\mathbf{W}^{[l]}$ uses random values and $\mathbf{b}^{[l]}$ starts at zero

2 linear_forward()

Computes $\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$
Combine inputs with weights, save the cache

3 linear_activation_forward()

Computes $\mathbf{A}^{[l]} = g^{[l]}(\mathbf{Z}^{[l]})$
Add non-linearity (ReLU or sigmoid)

4 L_model_forward()

Get predictions $\mathbf{A}^{[L]}$ from the input X
Returns $\mathbf{A}^{[L]}$ (probabilities) and the L caches

5 compute_cost()

Computes $J = -\frac{1}{m}\sum_{i}\left(y_i \log a_i + (1-y_i)\log(1-a_i)\right)$
Measure how wrong the predictions are



The 9 Core Functions

6 linear_backward()

$dW = A_{prev}^T \, dZ / m$, $db = \Sigma \, dZ / m$, $dA_{prev} = dZ \, W^T$
Get dW and db, and pass dA_prev back

7 linear_activation_backward()

Compute: $d\mathbf{Z}^{[l]} = d\mathbf{A}^{[l]} \odot g^{[l]\prime}(\mathbf{Z}^{[l]})$
Udo the activation to get dZ

8 L_model_backward()

dZ[L] = AL − Y returns every dW and db
Get the gradients of every layer

9 update_parameters()

W ← W − α·dW · b ← b − α·db
lower the cost by adjusting W and b



2. Initialization

What is initialization?

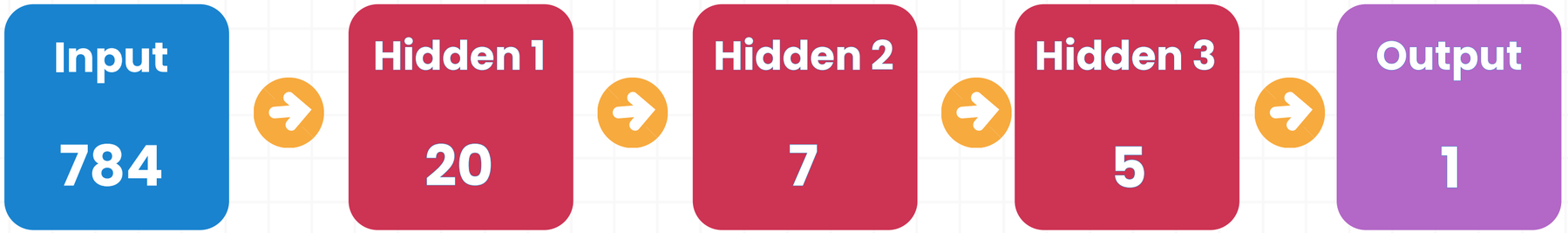
Initialization is the process of giving the neural network its starting parameters before training.

Each layer has two important parameters:

- **Weights (W):** randomly initialized values
- **Biases (b):** initialized to zero

Random weights help different neurons learn different things.

Example: layer_dims = [784, 20, 7, 5, 1]



Weight and Bias Initialization

How do we initialize parameters?

Parameter	Initialization	Shape
Weight $W^{[l]}$	Random values	$(n^{[l-1]}, n^{[l]})$
Bias $b^{[l]}$	Zeros	$(1, n^{[l]})$

Why random weights?

- Random values break symmetry between neurons.
- Different neurons can learn different features.

Why zero biases?

- Zero initialization is fine for biases.
- Random weights already break the symmetry

```
params[f"W{l}"] = torch.randn(n_prev, n_l) * s
params[f"b{l}"] = torch.zeros(1, n_l)
```

Why Do We Need a Scaling Factor?

The Scale

- For a 2-layer network, multiplying random weights by 0.01 was sufficient.
- For deeper networks, we use:

$$s = \sqrt{\frac{1}{n^{[l-1]}}} \qquad W^{[l]} = \text{Random values} \times s$$

Purpose: Keep the values of Z at a reasonable size from one layer to the next.

Output

```
W1: (784, 20)  std 0.0359
b1: (1, 20)
W2: (20, 7)    std 0.2253
b2: (1, 7)
W3: (7, 5)     std 0.3588
b3: (1, 5)
W4: (5, 1)     std 0.3239
b4: (1, 1)
```

- 4 weight matrices and 4 bias matrices are initialized.
- Each matrix has the correct dimensions.
- Weights contain random values.
- Biases are initialized to zero.

3 Forward Propagation

3.1 The linear step

$$\mathbf{Z}^{[l]} = \mathbf{A}^{[l-1]}\mathbf{W}^{[l]} + \mathbf{b}^{[l]}$$

- $A^{[l-1]}$: Previous layer's output
- $W^{[l]}$: Weight matrix
- $b^{[l]}$: Bias
- $Z^{[l]}$: Linear output

Example: $(3,4) \times (4,2) + (1,2) = (3,2)$

The **cache** is used to store temporary intermediate values computed during the forward propagation pass so that they can be easily reused during the backward propagation pass.

Cache: (A_prev, W, b)
Saved now, because backpropagation needs them later.

Code

```
def linear_forward(A_prev, W, b):
    Z = A_prev @ W + b
    cache = (A_prev, W, b)
    return Z, cache
torch.manual_seed(0)
A_prev = torch.randn(3, 4)
W = torch.randn(4, 2)
b = torch.tensor([[0.5, -0.5]])
Z, cache = linear_forward(A_prev, W, b)
print("Z =\n", Z)
print("shape of Z:", tuple(Z.shape))
```

Output

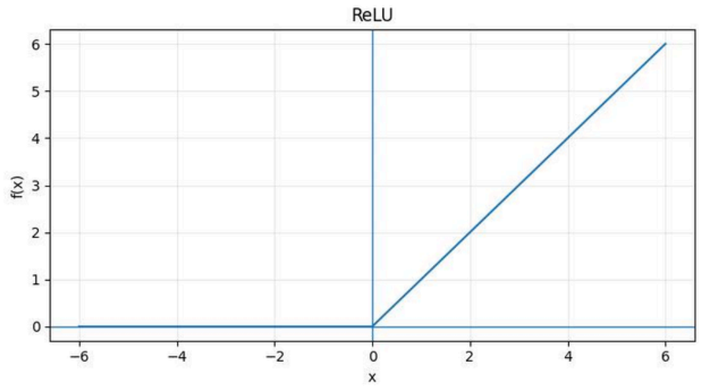
```
Z =
 tensor([[ 0.2210, -0.5107],
        [ 1.9513, -2.9794],
        [ 1.7237, -1.8384]])
shape of Z: (3, 2)
```



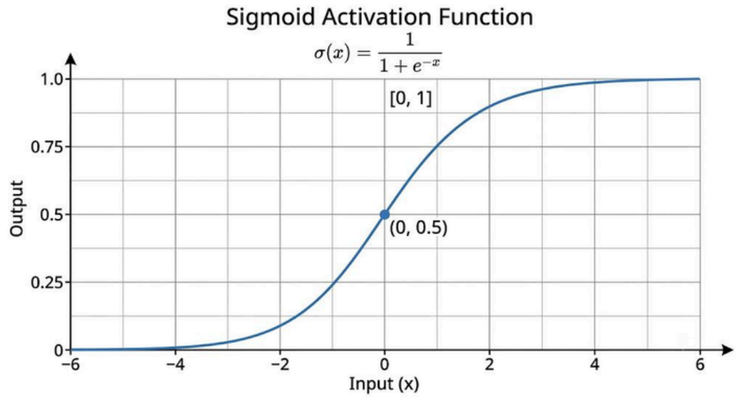
3.2 The activation

Purpose

sigmoid and **relu** return two things: the activation A, and Z as a cache, because the derivative g′(Z) is needed in the backward pass.



Negative values become 0; positive values remain unchanged.



Converts values into a range between 0 and 1.

How they are used?

- ReLu → Hidden Layer
- Sigmoid → Output Layer (binary classification)
- Both return activation A and cache Z.

3.2 The activation

Code:

```python
def sigmoid(Z):
    return torch.sigmoid(Z), Z          # (A, cache); the cache is Z

def relu(Z):
    return torch.clamp(Z, min=0), Z     # max(0, Z), elementwise

def linear_activation_forward(A_prev, W, b, activation):
    Z, linear_cache = linear_forward(A_prev, W, b)
    if activation == "sigmoid":
        A, activation_cache = sigmoid(Z)
    elif activation == "relu":
        A, activation_cache = relu(Z)
    return A, (linear_cache, activation_cache)

for activation in ["relu", "sigmoid"]:
    A, cache = linear_activation_forward(A_prev, W, b, activation)
    print(f"{activation}:\n", A)
```

Both functions return A and Z: g′(Z) is needed in the backward pass.

Compare result:

```
Z =
 tensor([[ 0.2210, -0.5107],
         [ 1.9513, -2.9794],
         [ 1.7237, -1.8384]])
```

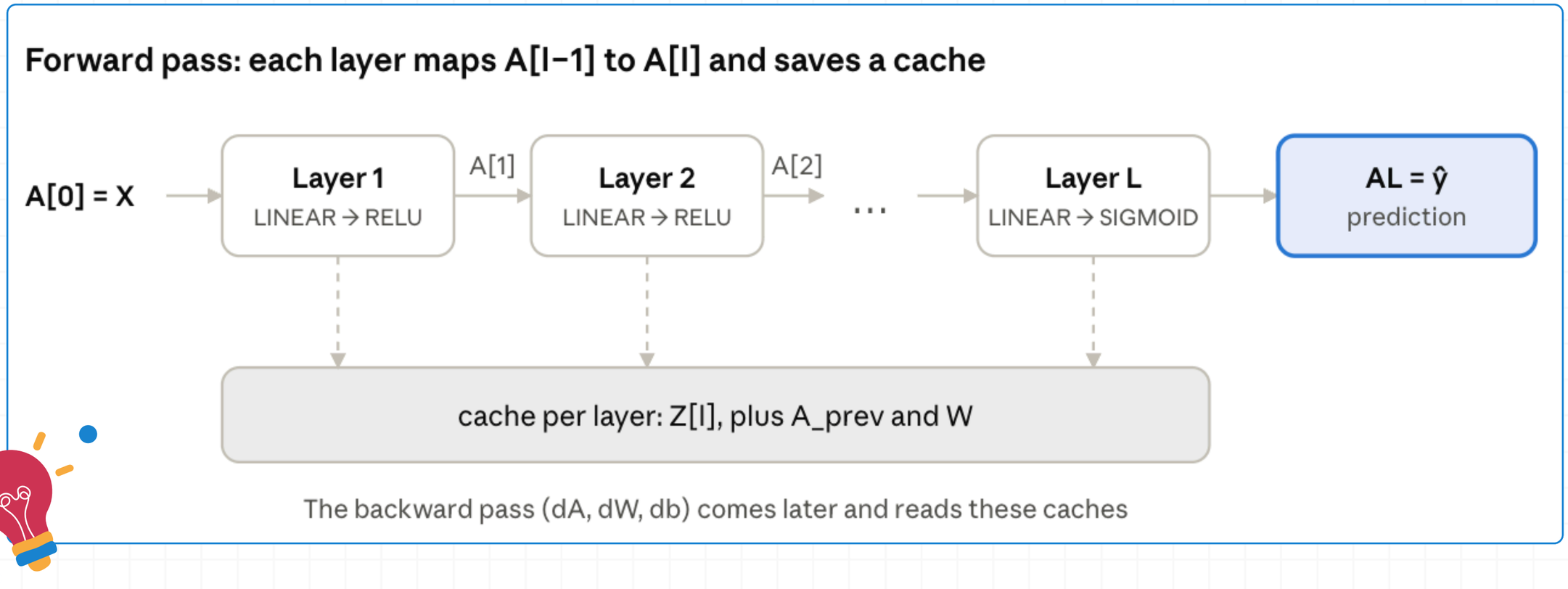
```
relu:
 tensor([[0.2210, 0.0000],
         [1.9513, 0.0000],
         [1.7237, 0.0000]])
```

```
sigmoid:
 tensor([[0.5550, 0.3750],
         [0.8756, 0.0484],
         [0.8486, 0.1372]])
```



3.3 The whole forward pass

Chain the layers: **ReLU** for every hidden layers, and **sigmoid** for the last one.



3.3 The whole forward pass

Chain the layers: **ReLU** for every hidden layers, and **sigmoid** for the last one.

Loop layers 1 to L-1 1
Linear → ReLU, saving a cache

Last layer 2
Linear → Sigmoid, a probability

Returns 3
AL and the list of caches





Code:

```python
def L_model_forward(X, params):
    # [LINEAR -> RELU] x (L - 1), then LINEAR -> SIGMOID.
    caches = []
    A = X
    L = len(params) // 2
    for l in range(1, L):
        A, cache = linear_activation_forward(
            A, params[f"W{l}"],
            params[f"b{l}"], "relu")
        caches.append(cache)
    AL, cache = linear_activation_forward(
        A, params[f"W{L}"],
        params[f"b{L}"], "sigmoid")
    caches.append(cache)
    return AL, caches
```

```python
torch.manual_seed(1)
X_small = torch.randn(6, 4)                          # m = 6 examples, 4 features
params_small = initialize_parameters_deep([4, 5, 3, 1])
AL, caches = L_model_forward(X_small, params_small)
print("AL =", AL.flatten())
print("shape of AL:", tuple(AL.shape), "| number of caches:", len(caches))
for l, ((A_in, W_l, b_l), Z_l) in enumerate(caches, start=1):
    print(f"layer {l}: A_prev {str(tuple(A_in.shape)):7s} W {str(tuple(W_l.shape)):7s} Z {str(tuple(Z_l.shape))}")
```

Output:

Checking the shapes: **X** of shape (6, 4) with layers [4, 5, 3, 1] – 6 examples, 4 features

```
AL = tensor([0.6058, 0.5583, 0.4515, 0.5379, 0.4829, 0.5692])
shape of AL: (6, 1) | number of caches: 3
layer 1: A_prev (6, 4)  W (4, 5)  Z (6, 5)
layer 2: A_prev (6, 5)  W (5, 3)  Z (6, 3)
layer 3: A_prev (6, 3)  W (3, 1)  Z (6, 1)
```

recap

Putting it all together

1 **Initialize**

random W, zero b

2 **Linear**

Z = A·W + b

3 **Activation**

ReLU or sigmoid

4 **Repeat**

layers 1 to L-1

5 **Probability**

AL between 0 and I



the end

Thank You Very Much!

